

Fundamental computer engineering assignment1

Yuxuan Wei

September 2026

Problem 1

Merging Overlapping Intervals

Problem Analysis

Goal

Combine overlapping intervals into non-overlapping intervals that preserve the same covered ranges.

Input

An array of intervals [start, end].

Output

Merged intervals ordered by starting value.

Basic Conditions

Each interval satisfies $\text{start} \leq \text{end}$. In this implementation, intervals sharing an endpoint are merged.

Problem 1

Algorithm Approach

Bubble Sort and Interval Merging

My approach is to first use bubble sort to sort the intervals in ascending order by comparing the first number of each interval.

Then, I compare the second number of the current interval with the first number of the next interval. If the second number of the current interval is greater than or equal to the first number of the next interval, I merge the two intervals: I replace the first number of the next interval with the first number of the current interval, set its second number to the larger of the two intervals' second numbers, and delete the current interval.

I then continue comparing the merged interval with the following interval. Otherwise, I keep both intervals unchanged and move on to compare the next interval with the one after it. I repeat this process until I reach the last interval.

Here, I assume that two intervals can also be merged when one ends exactly where the other begins.

Problem 1

Pseudocode

Algorithm 1 Merge Overlapping Intervals

Input: An array A of valid intervals [start, end].

Output: A updated in place to contain sorted, non-overlapping intervals.

```
1:  n ← SIZEOF(A)
2:  for remaining ← n downto 2 do
3:      swapped ← false
4:      for j ← 0 to remaining - 2 do
5:          if A[j].start > A[j + 1].start then
6:              SWAP(A[j], A[j + 1])
7:              swapped ← true
8:          end if
9:      end for
10:     if swapped = false then
11:         break
12:     end if
13: end for
14: i ← 0
15: while i + 1 < SIZEOF(A) do
16:     if A[i].end ≥ A[i + 1].start then
17:         A[i + 1].start ← A[i].start
18:         A[i + 1].end ← MAX(A[i].end, A[i + 1].end)
19:         DELETE(A, i)
20:     else
21:         i ← i + 1
22:     end if
23: end while
24: return A
```

Problem 1

Correctness Explanation

After Bubble Sort, the intervals are ordered by their starting values. The program then compares each interval with the next interval in this order.

If two intervals overlap, replacing them with one interval from the smaller start to the larger end keeps exactly the same covered values. If they do not overlap, the current interval cannot overlap with any later interval because all later intervals start no earlier than the next one.

Therefore, every overlap is merged and every remaining interval is separate from the next one. Empty input returns an empty result, and one interval is returned unchanged.

Problem 1

Time Complexity and Space Complexity

Let n be the number of intervals. The terms are worst-case counts. Each comparison, assignment, and index update takes $O(1)$ time.

1. Bubble-sort phase

Operation	Contribution
Initialization and return	2
Outer-loop control	$2n$
Inner-loop control	$n(n - 1)$
Compare starts and possible swap	$5n(n - 1)/2$

$$T_{\text{sort}}(n) \leq 2 + 2n + n(n - 1) + 5n(n - 1)/2$$

$$T_{\text{sort}}(n) \leq 7n^2/2 - 3n/2 + 2 \Rightarrow O(n^2)$$

2. Merge phase

Operation	Contribution
Scan-loop control and comparisons	$4n$
Erase after an overlap	$n(n - 1)/2$ shifts

$$T_{\text{merge}}(n) = O(n^2)$$

$$T(n) = O(n) + O(n^2) + O(n^2) \Rightarrow O(n^2)$$

Space Complexity

Only indices, a flag, and one temporary interval are used: auxiliary space is $O(1)$. Including the input vector, total storage is $O(n)$.

Problem 1

Test Cases

Case / Purpose	Input	Expected Output
1. Unsorted; overlap; touching; separate groups	[[8,10], [2,6], [1,3], [10,12]]	[[1,6], [8,12]]
2. Containment; duplicates	[[1,8], [2,3], [1,8]]	[[1,8]]
3. Empty input (edge)	[]	[]

Problem 1

Execution Screenshot

This VS Code debug run uses Test Case 1. The program merges the overlapping and touching intervals correctly and exits with status 0.

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem1/problem1'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem1'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem1/problem1'
Problem 1: Merge Overlapping Intervals
Enter the number of intervals: 4
Interval 1 (start end): 8 10
Interval 2 (start end): 2 6
Interval 3 (start end): 1 3
Interval 4 (start end): 10 12
Input: [[8, 10], [2, 6], [1, 3], [10, 12]]
Merged intervals: [[1, 6], [8, 12]]
Process exited with status 0
```

Input: [[8, 10], [2, 6], [1, 3], [10, 12]] Output: [[1, 6], [8, 12]]

Problem 2

Two Approaches to Sorting an Array of 0s, 1s, and 2s

Problem Analysis

Goal

Sort an array containing only 0, 1, and 2 in ascending order using two required approaches.

Input

An array of n values, where each value is 0, 1, or 2.

Output

The same array sorted in place with all 0s, then 1s, then 2s.

Basic Conditions

The report implements counting-based sorting and the one-pass Dutch National Flag algorithm.

Problem 2

Algorithm Approaches

1. Counting-Based Solution

My approach is to first initialize three integers, *a*, *b*, and *c*, to zero. They represent the numbers of 0s, 1s, and 2s, respectively. I then traverse the array using an index *i*. Whenever I encounter a 0, I increase *a* by one; whenever I encounter a 1, I increase *b* by one; and whenever I encounter a 2, I increase *c* by one.

After traversing the array, I start from its first position and set the first *a* elements to 0, the next *b* elements to 1, and the last *c* elements to 2. Finally, I print the modified array. This updates the original array in place instead of only printing the values in sorted order.

2. Dutch National Flag Algorithm

My approach is to first obtain the number of elements in the array, *n*. I then initialize three indices, *left*, *i*, and *right*, to 0, 0, and *n* - 1, respectively. I use *i* to examine the array.

When I encounter a 0, I swap it with the element at *left*, then increase both *left* and *i* by one. When I encounter a 1, I leave it unchanged and increase *i* by one. When I encounter a 2, I swap it with the element at *right* and decrease *right* by one. In this case, *i* stays unchanged so that I can examine the element that was swapped in.

I repeat these steps until *i* is greater than *right*. At that point, all elements have been processed, and the original array is sorted with all 0s first, followed by all 1s, and then all 2s. This method completes the sorting in one pass without using `std::sort()`.

Problem 2

Pseudocode

Algorithm 1 Counting-Based Sort

Input: An array A containing only 0, 1, and 2.

Output: A sorted in place in ascending order.

```
1:  $n \leftarrow \text{SIZEOF}(A)$ 
2:  $a \leftarrow 0; b \leftarrow 0; c \leftarrow 0$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:     if  $A[i] = 0$  then
5:          $a \leftarrow a + 1$ 
6:     else if  $A[i] = 1$  then
7:          $b \leftarrow b + 1$ 
8:     else
9:          $c \leftarrow c + 1$ 
10:    end if
11: end for
12:  $i \leftarrow 0$ 
13: for  $j \leftarrow 0$  to  $a - 1$  do
14:      $A[i] \leftarrow 0$ 
15:      $i \leftarrow i + 1$ 
16: end for
17: for  $j \leftarrow 0$  to  $b - 1$  do
18:      $A[i] \leftarrow 1$ 
19:      $i \leftarrow i + 1$ 
20: end for
21: for  $j \leftarrow 0$  to  $c - 1$  do
22:      $A[i] \leftarrow 2$ 
23:      $i \leftarrow i + 1$ 
24: end for
25: return  $A$ 
```

Problem 2

Pseudocode

Algorithm 2 Dutch National Flag

Input: An array A containing only 0, 1, and 2.

Output: A sorted in place in ascending order.

```
1:  $n \leftarrow \text{SIZEOF}(A)$ 
2:  $\text{left} \leftarrow 0; i \leftarrow 0; \text{right} \leftarrow n - 1$ 
3: while  $i \leq \text{right}$  do
4:     if  $A[i] = 0$  then
5:          $\text{SWAP}(A[i], A[\text{left}])$ 
6:          $\text{left} \leftarrow \text{left} + 1$ 
7:          $i \leftarrow i + 1$ 
8:     else if  $A[i] = 1$  then
9:          $i \leftarrow i + 1$ 
10:    else
11:         $\text{SWAP}(A[i], A[\text{right}])$ 
12:         $\text{right} \leftarrow \text{right} - 1$ 
13:    end if
14: end while
15: return  $A$ 
```

Problem 2

Counting-Based Solution - Correctness Explanation

The first loop examines every array value once and increases exactly one of the three counters. Therefore, a , b , and c equal the numbers of 0s, 1s, and 2s in the original array.

The next three loops write a zeros, then b ones, and then c twos. Because $a + b + c$ equals the array length, every position is written once and the result is sorted.

The result contains the same number of 0s, 1s, and 2s as the input. If the array is empty, all loops perform zero iterations and the empty array is returned.

Problem 2

Counting-Based Solution - Time and Space

Let n be the array length. The terms are worst-case counts. Each comparison, assignment, and index update takes $O(1)$ time.

1. Count values

Operation	Contribution
Initialize a, b, and c	3
For-loop control	$2n + 1$
If/else test and counter update	$3n$

$$T_{\text{count}}(n) \leq 3 + (2n + 1) + 3n = 5n + 4 \Rightarrow O(n)$$

2. Write values back

Operation	Contribution
Three loop controls	$2n + 3$
Write value and advance index	$2n$

$$T_{\text{write}}(n) \leq 4n + 3$$

$$T(n) \leq (5n + 4) + (4n + 3) = 9n + 7 \Rightarrow O(n)$$

Space Complexity

Only three counters and loop indices are needed, so auxiliary space is $O(1)$. Including the input vector, total storage is $O(n)$.

Problem 2

Dutch National Flag - Correctness Explanation

The pointer `left` marks the next position for a 0, `i` marks the current value, and `right` marks the next position for a 2. When the current value is 0, it is moved to the left region. When it is 1, it remains in the middle region. When it is 2, it is moved to the right region.

Each iteration classifies one value by increasing `i` or decreasing `right`. Thus, when the loop ends, all values have been placed into the order 0s, then 1s, then 2s.

Only swaps are used, so no input values are lost or added. An empty array has no unclassified values and is already sorted.

Problem 2

Dutch National Flag - Time and Space

Let n be the array length. The unclassified region loses exactly one element in every while-loop iteration.

1. While loop

Operation	Contribution
Initialize left, i, and right	3
While-loop control	$2n + 1$
If/else test, swap, and index updates	$6n$

$$T(n) \leq 3 + (2n + 1) + 6n = 8n + 4 \Rightarrow O(n)$$

2. General cases

Operation	Contribution
Nested loops	0
Recursion	0

Best, average, and worst cases are linear because exactly n elements are classified.

Space Complexity

The method uses left, i, right, and a constant-size swap temporary: auxiliary space is $O(1)$. Including the input vector, total storage is $O(n)$.

Problem 2

Test Cases

Run each case with both Counting Sort and Dutch National Flag.

Case / Purpose	Input	Expected Output
1. All categories; unequal counts; unsorted	[2,0,1,2,0,2]	[0,0,1,2,2,2]
2. One category; repeated swaps (edge)	[2,2,2]	[2,2,2]
3. Empty input (edge)	[]	[]

Problem 2

Execution Screenshot

This VS Code debug run uses the Counting-based solution for Test Case 1. The program sorts the input values correctly and exits with status 0.

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem2/problem2'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem2'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem2/problem2'
Problem 2: Sort 0, 1, and 2
Choose a method:
1. Counting-based solution
2. Dutch National Flag (one pass)
Your choice: 1
Enter the number of elements: 6
Enter 6 values (only 0, 1, or 2):
2 0 1 2 0 2
Input: [2, 0, 1, 2, 0, 2]
Method: Counting-based solution
Sorted array: [0, 0, 1, 2, 2, 2]
Process exited with status 0
```

Input: [2, 0, 1, 2, 0, 2] Output: [0, 0, 1, 2, 2, 2]

Problem 3

Largest Number

Problem Analysis

Goal

Rearrange nonnegative integers so that their decimal representations form the largest possible concatenated number.

Input

A vector of nonnegative integers.

Output

A string containing the largest concatenated number.

Basic Conditions

Numbers are compared through their string concatenations. If all values are zero, the output is "0".

Problem 3

Algorithm Approach

Bubble Sort with Concatenation Comparison

My approach is to first convert every number in the array into a string. I then use bubble sort and an index i to compare adjacent strings A and B .

I compare their two possible concatenations, AB and BA . If AB is smaller than BA , I swap A and B ; otherwise, I keep their order unchanged. I repeat these comparisons and swaps until the array is sorted according to this rule. If a full pass makes no swaps, I stop sorting early.

Finally, I join all the sorted strings in order and return the result as a string. If all numbers are zero, I return only "0". For an empty input array, I return an empty string.

Problem 3

Pseudocode

Algorithm 1 Largest Number Using Bubble Sort

Input: A vector V of nonnegative integers.

Output: A string representing the largest concatenated number.

```
1:  n ← SIZEOF(V)
2:  if n = 0 then
3:      return ""
4:  end if
5:  S ← an empty array of strings
6:  for i ← 0 to n - 1 do
7:      APPEND(S, TO_STRING(V[i]))
8:  end for
9:  for remaining ← n downto 2 do
10:     swapped ← false
11:     for i ← 0 to remaining - 2 do
12:         AB ← CONCAT(S[i], S[i + 1])
13:         BA ← CONCAT(S[i + 1], S[i])
14:         if AB < BA then
15:             SWAP(S[i], S[i + 1])
16:             swapped ← true
17:         end if
18:     end for
19:     if swapped = false then
20:         break
21:     end if
22: end for
23: if S[0] = "0" then
24:     return "0"
25: end if
26: result ← ""
27: for i ← 0 to n - 1 do
28:     APPEND_DIGITS(result, S[i])
29: end for
30: return result
```

Problem 3

Correctness Explanation

For any two adjacent strings A and B, the program compares AB with BA. If BA is larger, swapping A and B makes the concatenated result larger at that position.

Bubble Sort repeats this comparison for every adjacent pair. When no swap is needed, every adjacent pair is in the order that gives the larger concatenation.

Therefore, joining the final string order produces the largest possible number. If all values are zero, the program returns the single string "0"; empty input returns an empty result.

Problem 3

Time Complexity and Space Complexity

Let n be the number of integers and d be the maximum digit count. The terms are worst-case counts. Each comparison, assignment, and index update takes $O(1)$ time.

1. Conversion and comparison work

Operation	Contribution
Convert integers to strings	nd
Build and compare AB and BA	$6d$ per comparison

$$T_{\text{convert}}(n, d) = nd \Rightarrow O(nd)$$

2. Bubble-sort phase

Operation	Contribution
Outer-loop control	$2n$
Inner-loop control	$n(n - 1)$
Comparator work	$3d n(n - 1)$

$$T_{\text{sort}}(n, d) = O(n^2d)$$

$$T(n, d) = O(nd) + O(n^2d) + O(nd) \Rightarrow O(n^2d)$$

For fixed-size C++ integers, d is constant, so $T(n) = O(n^2)$.

Space Complexity

The string array and returned result store $O(nd)$ characters. Temporary concatenations use $O(d)$ space, so total storage is $O(nd)$; with fixed-size integers, it is $O(n)$.

Problem 3

Test Cases

Case / Purpose	Input	Expected Output
1. Prefix ordering; duplicates; zero	[12,121,12,0]	"12121210"
2. Different lengths; numeric vs. custom order	[3,30,34,5,9]	"9534330"
3. All zeros (edge)	[0,0,0]	"0"

Problem 3

Execution Screenshots

Both VS Code debug runs exited with status 0. They verify the concatenation comparison for prefix ordering and for values with different digit lengths.

Test Case 1: Prefix ordering, duplicates, and zero

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3/problem3'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3/problem3'
Problem 3: Largest Number
Enter the number of elements: 4
Enter 4 nonnegative integers:
12 121 12 0
Input: [12, 121, 12, 0]
Largest number (string): "12121210"
Process exited with status 0
```

Test Case 2: Different digit lengths

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3/problem3'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem3/problem3'
Problem 3: Largest Number
Enter the number of elements: 5
Enter 5 nonnegative integers:
3 30 34 5 9
Input: [3, 30, 34, 5, 9]
Largest number (string): "9534330"
Process exited with status 0
```

Problem 4

Group Anagrams

Problem Analysis

Goal

Place words with the same letter-frequency pattern into the same group.

Input

A vector of lowercase English words.

Output

A collection of anagram groups containing every input word exactly once.

Basic Conditions

The implementation uses a 26-counter signature for each word and keeps the original order within each group.

Problem 4

Algorithm Approach

Letter Counting and Sequential Group Comparison

My approach is to first traverse all the words using an index. For each word, I create an array of 26 numbers and initialize all of them to zero. From the first position to the last, these numbers represent the letters a, b, ..., z.

I then examine every letter in the word. Whenever I encounter an a, I increase the first number by one. If a appears more than once, I increase that number each time. I handle all other letters in the same way.

After processing the first word, I store its count array and create a group containing that word. For each following word, I repeat the counting process with a new zero-initialized array and compare it with the stored arrays one by one. If I find an identical array, I append the word to the corresponding group. Otherwise, I store the new count array and create a new group.

I process the words in their original input order and always append matching words to the end of their groups. This completes the grouping while preserving the original order within each group. Here, I assume that all words contain only lowercase English letters a-z.

Problem 4

Pseudocode

Algorithm 1 Group Anagrams Using Letter Counts

Input: A vector Words of strings containing only lowercase letters a-z.

Output: Anagram groups preserving input order within each group.

```
1: Signatures ← an empty list
2: Groups ← an empty list
3: for each word in Words, in input order do
4:     Counts ← an array of 26 zeros
5:     for each letter in word do
6:         k ← POSITION(letter in 'a'..'z', starting at 0)
7:         Counts[k] ← Counts[k] + 1
8:     end for
9:     found ← false
10:    for j ← 0 to SIZEOF(Signatures) - 1 do
11:        if Counts = Signatures[j] then
12:            APPEND(Groups[j], word)
13:            found ← true
14:            break
15:        end if
16:    end for
17:    if found = false then
18:        APPEND(Signatures, Counts)
19:        APPEND(Groups, [word])
20:    end if
21: end for
22: return Groups
```

Problem 4

Correctness Explanation

For each word, the 26-counter array records how many times each letter appears. Two words have the same counter array exactly when they contain the same letters with the same frequencies, so they are anagrams.

The program compares each new counter array with the stored group signatures. A matching signature places the word in the correct existing group; otherwise a new group is created.

Every input word is processed once and added to exactly one group. Therefore, all and only anagrams are grouped together. Empty input produces no groups.

Problem 4

Time Complexity and Space Complexity

Let n be the number of words and L be the total number of characters. Each signature has 26 counters, which is a constant.

1. Build word signatures

Operation	Contribution
Initialize 26 counters	$26n$
Read one character and increment	$2L$

$$T_{\text{signature}}(n, L) = 26n + 2L \Rightarrow O(n + L)$$

2. Search existing groups

Operation	Contribution
Compare two signatures	$26n(n - 1)/2$
Copy words into their groups	$O(n + L)$

$$T(n, L) \leq 26n + 2L + 26n(n - 1)/2 + O(n + L)$$

$$T(n, L) = O(n^2 + L)$$

Space Complexity

Stored signatures use $O(n)$ space. The returned groups store $O(n + L)$ data, so total storage including input and output is $O(n + L)$.

Problem 4

Test Cases

Group order may vary; word order within each group must match the input.

Case / Purpose	Input	Expected Output
1. Interleaved groups; duplicates; stable order	["ab","cat","ba","ab","act"]	[["ab","ba","ab"], ["cat","act"]]
2. Repeated letters; unequal lengths; singleton	["aab","aba","ab","baa"]	[["aab","aba","baa"], ["ab"]]
3. Empty input (edge)	[]	[]

Problem 4

Execution Screenshots

Both VS Code debug runs exit with status 0. They verify that anagrams are grouped together and that the input order is preserved within each group.

Test Case 1: Interleaved groups and duplicate words

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4/problem4'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4/problem4'
Problem 4: Group Anagrams
Enter the number of words: 5
Enter one word per line (lowercase a-z only).
A blank line represents an empty string.
Word 1: ab
Word 2: cat
Word 3: ba
Word 4: ab
Word 5: act
Input: ["ab", "cat", "ba", "ab", "act"]
Anagram groups: [["ab", "ba", "ab"], ["cat", "act"]]
Process exited with status 0
```

Test Case 2: Repeated letters, unequal lengths, and a singleton group

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4/problem4'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem4/problem4'
Problem 4: Group Anagrams
Enter the number of words: 4
Enter one word per line (lowercase a-z only).
A blank line represents an empty string.
Word 1: aab
Word 2: aba
Word 3: ab
Word 4: baa
Input: ["aab", "aba", "ab", "baa"]
Anagram groups: [["aab", "aba", "baa"], ["ab"]]
Process exited with status 0
```

Problem 5

Meeting Attendance

Problem Analysis

Goal

Determine whether one person can attend every meeting without time conflicts.

Input

A vector of meeting intervals [start, end].

Output

Return true when no meetings overlap; otherwise return false.

Basic Conditions

Meeting endpoints are integer values. A meeting may begin when another meeting ends.

Problem 5

Algorithm Approach

Bubble Sort and Adjacent Meeting Comparison

My approach is to first use bubble sort to arrange the meeting intervals in ascending order by comparing the first number of each interval, which represents its starting time.

I then use an index i to traverse the sorted array and compare the second number of the current meeting with the first number of the next meeting. If the second number of the current meeting is greater than the first number of the next meeting, the meetings overlap, so I immediately return false, meaning that one person cannot attend every meeting.

Otherwise, I increase i by one and continue comparing the next pair of adjacent meetings. If I reach the end without finding an overlap, I return true. Equal endpoints do not count as an overlap. If the array is empty or contains only one meeting, I also return true.

Problem 5

Pseudocode

Algorithm 1 Check Meeting Attendance

Input: An array M of meeting intervals with $\text{start} < \text{end}$.

Output: true if all meetings can be attended; otherwise false.

```
1:  n ← SIZEOF(M)
2:  for remaining ← n downto 2 do
3:      swapped ← false
4:      for i ← 0 to remaining - 2 do
5:          if M[i].start > M[i + 1].start then
6:              SWAP(M[i], M[i + 1])
7:              swapped ← true
8:          end if
9:      end for
10:     if swapped = false then
11:         break
12:     end if
13: end for
14: for i ← 0 to n - 2 do
15:     if M[i].end > M[i + 1].start then
16:         return false
17:     end if
18: end for
19: return true
```

Problem 5

Correctness Explanation

After Bubble Sort, meetings are ordered by starting time. The program only needs to compare adjacent meetings in this order.

If a meeting ends after the next meeting starts, the two meetings overlap and the program correctly returns false. If every adjacent pair does not overlap, no earlier meeting can overlap with a later meeting because starting times are ordered.

Therefore, the program returns true exactly when all meetings can be attended. Empty input and one meeting have no conflict, so the result is true.

Problem 5

Time Complexity and Space Complexity

Let n be the number of meetings. The terms are worst-case counts. Each comparison, assignment, and index update takes $O(1)$ time.

1. Bubble-sort phase

Operation	Contribution
Initialization and return	2
Outer-loop control	$2n$
Inner-loop control	$n(n - 1)$
Compare starts and possible swap	$5n(n - 1)/2$

$$T_{\text{sort}}(n) \leq 2 + 2n + n(n - 1) + 5n(n - 1)/2$$

$$T_{\text{sort}}(n) \leq 7n^2/2 - 3n/2 + 2 \Rightarrow O(n^2)$$

2. Overlap-check phase

Operation	Contribution
Loop control and adjacent comparison	$4(n - 1)$

$$T_{\text{check}}(n) = O(n)$$

$$T(n) = O(n) + O(n^2) + O(n) \Rightarrow O(n^2)$$

Space Complexity

Sorting and checking use a constant number of variables and one temporary meeting: auxiliary space is $O(1)$. Including the input vector, total storage is $O(n)$.

Problem 5

Test Cases

true means all meetings can be attended; false means an overlap exists.

Case / Purpose	Input	Expected Output
1. Unsorted; touching; separated meetings	[[8,10], [3,5], [1,3]]	true
2. Unsorted; containment; overlap	[[4,6], [1,8], [9,10]]	false
3. Empty input (edge)	[]	true

Problem 5

Execution Screenshots

Both VS Code debug runs exited with status 0. The first test confirms that meetings with touching endpoints can all be attended. The second test confirms that an overlapping meeting prevents attendance of all meetings.

Test Case 1: Touching endpoints are allowed

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5/problem5'  
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5'  
1 arguments:  
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5/problem5'  
Problem 5: Meeting Attendance  
Enter the number of meetings: 3  
Meeting 1 (start end): 8 10  
Meeting 2 (start end): 3 5  
Meeting 3 (start end): 1 3  
Input: [[8, 10], [3, 5], [1, 3]]  
Can attend all meetings: true  
Process exited with status 0
```

Test Case 2: Containment creates an overlap

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5/problem5'  
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5'  
1 arguments:  
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem5/problem5'  
Problem 5: Meeting Attendance  
Enter the number of meetings: 3  
Meeting 1 (start end): 4 6  
Meeting 2 (start end): 1 8  
Meeting 3 (start end): 9 10  
Input: [[4, 6], [1, 8], [9, 10]]  
Can attend all meetings: false  
Process exited with status 0
```

Problem 6

K Closest Points to the Origin

Problem Analysis

Goal

Find the k points closest to the origin and compare a sorting approach with a heap-based approach.

Input

A vector of coordinate pairs (x, y) and an integer k with $0 \leq k \leq n$.

Output

Return k points closest to the origin.

Basic Conditions

Distances are compared with $x^2 + y^2$, which has the same ordering as Euclidean distance.

Problem 6

Algorithm Approaches

1. Sorting-Based Solution

My approach is to first use an index i to traverse all the points. For each point, I assign its first coordinate to x and its second coordinate to y , then calculate its squared distance from the origin using $d^2 = x^2 + y^2$.

I repeat this process for every point and sort all points in ascending order of squared distance, keeping each point together with its distance. Finally, I return the first k points. If k is zero, I return an empty result. The C++ implementation uses `std::sort()`.

2. Max-Heap Solution

My approach is to first create a max heap to keep the k closest candidate points, with the farthest candidate at the top. I traverse every point, calculate its squared distance $d^2 = x^2 + y^2$, and insert the point together with its distance into the heap.

If the heap contains more than k points, I remove the top point, which is the farthest candidate. I repeat these steps until all points have been processed. The remaining points are the k closest points. I then remove them one by one and reverse the result so it is ordered from nearest to farthest. If k is zero, I return an empty result.

Problem 6

Pseudocode

Algorithm 1 K Closest Points Using Sorting

Input: A list Points of coordinate pairs and k, with $0 \leq k \leq \text{SIZEOF}(\text{Points})$.

Output: The k closest points, ordered by distance, then x, then y.

```
1: if k = 0 then
2:     return an empty list
3: end if
4: Records ← an empty list
5: for each point (x, y) in Points do
6:     distanceSquared ←  $x \times x + y \times y$ 
7:     APPEND(Records, (x, y, distanceSquared))
8: end for
9: SORT Records by (distanceSquared, x, y) ascending
10: Result ← an empty list
11: for i ← 0 to k - 1 do
12:     APPEND(Result, Records[i].point)
13: end for
14: return Result
```

Problem 6

Pseudocode

Algorithm 2 K Closest Points Using a Max Heap

Input: A list Points of coordinate pairs and k, with $0 \leq k \leq \text{SIZEOF}(\text{Points})$.

Output: The k closest points, ordered by distance, then x, then y.

```
1: if k = 0 then
2:     return an empty list
3: end if
4: H ← an empty max heap ordered by (distanceSquared, x, y)
5: for each point (x, y) in Points do
6:     distanceSquared ← x × x + y × y
7:     INSERT(H, (x, y, distanceSquared))
8:     if SIZEOF(H) > k then
9:         REMOVE_MAX(H)
10:    end if
11: end for
12: Result ← an empty list
13: while H is not empty do
14:     record ← REMOVE_MAX(H)
15:     APPEND(Result, record.point)
16: end while
17: REVERSE(Result)
18: return Result
```

Problem 6

Correctness Explanation

Squared distance $x^2 + y^2$ has the same order as Euclidean distance, so comparing squared distances correctly identifies closer points.

In the sorting method, all points are ordered by this distance and the first k points are returned. In the heap method, each new point is added and the farthest retained point is removed when necessary, so the heap keeps the k closest points processed so far.

After every point has been processed, both methods return k closest points. When k is zero, both methods return an empty result.

Problem 6

Time Complexity and Space Complexity

Let n be the number of points and $0 \leq k \leq n$. Computing one squared distance is $O(1)$.

1. Sorting method

Operation	Contribution
Compute distance keys	$4n$
Comparison sort	$c n \log_2 n$
Copy the first k points	k

$$T_{\text{sort}}(n) \leq 4n + c n \log_2 n + k \Rightarrow O(n \log n)$$

2. Max-heap method

Operation	Contribution
Scan-loop control	$2n + 2$
Insert and possible removal	$2n \log_2(k + 1)$
Extract k retained points	$k \log_2(k + 1)$

$$T_{\text{heap}}(n, k) = O(n \log(k + 1)); \text{ for } k = 1, \text{ it is } O(n).$$

Space Complexity

The sorting method stores a copy of n points, so it uses $O(n)$ extra space. The heap method stores k points and an $O(k)$ result, giving $O(k)$ extra space.

Problem 6

Test Cases

Run both methods. Equal distances use the program's coordinate tie-break (x, then y).

Case / Purpose	Input	Expected Output
1. Origin; negatives; distance tie at cutoff	points = [[1,0], [0,0], [-1,0], [0,1]]; k = 3	[[0,0], [-1,0], [0,1]]
2. Duplicate points; k = n (edge)	points = [[2,0], [-1,0], [-1,0]]; k = 3	[[-1,0], [-1,0], [2,0]]
3. Zero requested (edge)	points = [[1,2]]; k = 0	[]

Problem 6

Execution Screenshots

Both VS Code debug runs exited with status 0. The screenshots verify the sorting method and the max-heap method for finding the k closest points to the origin.

Test Case 1: Sorting method

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6/problem6'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6/problem6'
Problem 6: K Closest Points to the Origin
1. Sorting
2. Max heap
Choose a method: 1
Enter the number of points: 4
Enter k (0 to 4): 3
Point 1 (x y): 1 0
Point 2 (x y): 0 0
Point 3 (x y): -1 0
Point 4 (x y): 0 1
Input: [[1, 0], [0, 0], [-1, 0], [0, 1]]
k: 3
Method: Sorting
Closest points: [[0, 0], [-1, 0], [0, 1]]
Process exited with status 0
```

Test Case 2: Max-heap method

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6/problem6'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem6/problem6'
Problem 6: K Closest Points to the Origin
1. Sorting
2. Max heap
Choose a method: 2
Enter the number of points: 3
Enter k (0 to 3): 3
Point 1 (x y): 2 0
Point 2 (x y): -1 0
Point 3 (x y): -1 0
Input: [[2, 0], [-1, 0], [-1, 0]]
k: 3
Method: Max heap
Closest points: [[-1, 0], [-1, 0], [2, 0]]
Process exited with status 0
```

Problem 7

Top K Frequent Values

Problem Analysis

Goal

Find the k most frequent distinct values in an integer array.

Input

An integer array and an integer k, where $0 \leq k \leq$ the number of distinct values.

Output

Return k distinct values in descending frequency order.

Basic Conditions

When frequencies are equal, the smaller value is placed first.

Problem 7

Algorithm Approach

Frequency Counting and Bubble Sort

My approach is to first traverse the entire array using an index i and create a table to store each distinct value and its frequency. If the current value is already in the table, I increase its frequency by one. Otherwise, I add the value to the table and set its frequency to 1.

After traversing the array, I use bubble sort to sort the records in the table. I compare the frequencies of adjacent values and place the value with the higher frequency first. If their frequencies are equal, I place the smaller value first. During sorting, I keep each value together with its frequency.

Finally, I read and return the first k values from the sorted table. If k is zero, I return an empty result. The value of k must not be negative or exceed the number of distinct values in the input array.

Problem 7

Pseudocode

Algorithm 1 Top K Frequent Values

Input: An integer array A and k, with $0 \leq k \leq$ number of distinct values.

Output: k values ordered by decreasing frequency, then increasing value.

```
1: Table ← an empty list
2: for each value in A do
3:     found ← false
4:     for each record in Table do
5:         if record.value = value then
6:             record.count ← record.count + 1
7:             found ← true
8:             break
9:         end if
10:    end for
11:    if found = false then
12:        APPEND(Table, (value, 1))
13:    end if
14: end for
15: if k = 0 then
16:     return an empty list
17: end if
18: for remaining ← SIZEOF(Table) downto 2 do
19:     swapped ← false
20:     for i ← 0 to remaining - 2 do
21:         if Table[i].count < Table[i + 1].count or
22:            (Table[i].count = Table[i + 1].count and
23:             Table[i].value > Table[i + 1].value) then
24:             SWAP(Table[i], Table[i + 1])
25:             swapped ← true
26:         end if
27:     end for
28:     if swapped = false then
29:         break
30:     end if
31: end for
32: Result ← an empty list
33: for i ← 0 to k - 1 do
34:     APPEND(Result, Table[i].value)
35: end for
36: return Result
```

Problem 7

Correctness Explanation

The frequency table is updated once for every input value. If the value already has a record, its count increases; otherwise a new record is created. Therefore, each record has the correct final frequency.

Bubble Sort orders the records by decreasing frequency. When two frequencies are equal, it places the smaller value first.

The first k records after sorting are therefore the k required values in the required order. If k is zero, the returned result is empty.

Problem 7

Time Complexity and Space Complexity

Let n be the input length and m be the number of distinct values, where $m \leq n$. The frequency table is searched linearly.

1. Frequency-count phase

Operation	Contribution
Search earlier frequency records	$n(n - 1)/2$
Update or insert frequency	$3n$

$$T_{\text{count}}(n) \leq n(n - 1)/2 + 3n \Rightarrow O(n^2)$$

2. Bubble-sort frequency records

Operation	Contribution
Loop controls and comparisons	$3m(m - 1)/2$
Output top k values	k

$$T_{\text{sort}}(m) = O(m^2) \leq O(n^2)$$

$$T(n) = O(n^2) + O(m^2) + O(k) \Rightarrow O(n^2)$$

Space Complexity

The frequency table stores m records and the result stores k values: $O(m + k)$ extra space, which is $O(n)$ because m and k do not exceed n .

Problem 7

Test Cases

Case / Purpose	Input	Expected Output
1. Different counts; tie at cutoff; negatives	A = [-1,2,2,3,-1,3,3]; k = 2	[3,-1]
2. All tied; all distinct values requested	A = [4,1,2,4,1,2]; k = 3	[1,2,4]
3. Empty input (edge)	A = []; k = 0	[]

Problem 7

Execution Screenshots

Both VS Code debug runs exited with status 0. The first test verifies the output when frequencies differ. The second test verifies the deterministic order when all selected values have the same frequency.

Test Case 1: Different frequencies

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7/problem7'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7/problem7'
Problem 7: Top K Frequent Values
Enter the number of elements: 7
Enter 7 integers:
-1 2 2 3 -1 3 3
Enter k (0 to 3): 2
Input: [-1, 2, 2, 3, -1, 3, 3]
k: 2
Top k frequent values: [3, -1]
Process exited with status 0
```

Test Case 2: Equal frequencies

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7/problem7'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem7/problem7'
Problem 7: Top K Frequent Values
Enter the number of elements: 6
Enter 6 integers:
4 1 2 4 1 2
Enter k (0 to 3): 3
Input: [4, 1, 2, 4, 1, 2]
k: 3
Top k frequent values: [1, 2, 4]
Process exited with status 0
```

Problem 8

Merge K Sorted Arrays

Problem Analysis

Goal

Merge k sorted integer arrays into one sorted array.

Input

A collection of k integer arrays, each sorted in nondecreasing order.

Output

One sorted array containing every input value.

Basic Conditions

The input arrays are not changed, and duplicate values are preserved.

Problem 8

Algorithm Approach

Separate Indices and Repeated Minimum Selection

My approach is to first create an index for each array and initialize every index to 0. Each index points to the first element in its array that has not yet been taken. I then create an empty result array.

I compare the values currently pointed to in all arrays, append the smallest value to the end of the result array, and increase only the index of the array from which that value was taken. The indices of the other arrays remain unchanged.

I repeat these comparisons and additions. If all elements of an array have already been taken, I skip that array in later comparisons. I continue until every element from every array has been taken, then return the result array.

Empty arrays are skipped. If all arrays are empty, I return an empty result. Here, I assume that every input array is already sorted in ascending order, allowing equal adjacent values.

Problem 8

Pseudocode

Algorithm 1 Merge K Sorted Arrays

Input: k integer arrays A , each sorted in nondecreasing order.

Output: One sorted array containing every input occurrence.

```
1:  $k \leftarrow \text{SIZEOF}(A)$ 
2:  $\text{Position} \leftarrow$  an array of  $k$  zeros
3:  $\text{Result} \leftarrow$  an empty list
4: while true do
5:      $\text{found} \leftarrow \text{false}$ 
6:      $\text{smallestArray} \leftarrow 0$ 
7:     for  $i \leftarrow 0$  to  $k - 1$  do
8:         if  $\text{Position}[i] = \text{SIZEOF}(A[i])$  then
9:             continue
10:        end if
11:        if  $\text{found} = \text{false}$  then
12:             $\text{smallestArray} \leftarrow i$ 
13:             $\text{found} \leftarrow \text{true}$ 
14:        else if  $A[i][\text{Position}[i]] <$ 
15:             $A[\text{smallestArray}][\text{Position}[\text{smallestArray}]]$  then
16:             $\text{smallestArray} \leftarrow i$ 
17:        end if
18:    end for
19:    if  $\text{found} = \text{false}$  then
20:        break
21:    end if
22:     $\text{APPEND}(\text{Result}, A[\text{smallestArray}][\text{Position}[\text{smallestArray}]])$ 
23:     $\text{Position}[\text{smallestArray}] \leftarrow \text{Position}[\text{smallestArray}] + 1$ 
24: end while
25: return  $\text{Result}$ 
```

Problem 8

Correctness Explanation

Each array index always points to the first value in that array that has not yet been copied. Because every input array is sorted, this pointed value is the smallest remaining value from its array.

The program selects the smallest available pointed value among all arrays, appends it to the result, and advances only the index of the selected array. Therefore, the appended value is the smallest value not yet placed in the result.

Repeating this step copies every input value once and produces a nondecreasing result. Empty arrays are skipped, and no arrays produce an empty result.

Problem 8

Time Complexity and Space Complexity

Let n be the total number of elements and k be the number of arrays. One candidate check is $O(1)$.

1. Initialization

Operation	Contribution
Create k position indices	k

$$T_{\text{init}}(k) = k \Rightarrow O(k)$$

2. Repeated minimum selection

Operation	Contribution
While-loop control	$n + 1$
Scan all array heads	$k(n + 1)$
Append value and advance index	$2n$

$$T_{\text{merge}}(n, k) \leq k + (n + 1) + k(n + 1) + 2n$$

$$T(n, k) = O(nk + n + k); \text{ for } n, k \geq 1, \text{ this is } O(nk).$$

Space Complexity

The position array uses $O(k)$ auxiliary space. The returned merged array stores n values, so extra storage including the output is $O(n + k)$.

Problem 8

Test Cases

Case / Purpose	Input	Expected Output
1. Empty member; negatives; duplicates; unequal lengths	[[], [-3,1,1], [-2,1], [0]]	[-3,-2,0,1,1,1]
2. One array; duplicates (edge)	[[-1,-1,2]]	[-1,-1,2]
3. No arrays (edge)	[]	[]

Problem 8

Execution Screenshots

Both VS Code debug runs exited with status 0. The first test verifies merging arrays when one input array is empty. The second test verifies the single-array case and preservation of duplicate values.

Test Case 1: One input array is empty

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8/problem8'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8/problem8'
Problem 8: Merge K Sorted Arrays
Enter the number of arrays: 4
Array 1 size: 0
Enter 0 integers in nondecreasing order:
Array 2 size: 3
Enter 3 integers in nondecreasing order:
-3 1 1
Array 3 size: 2
Enter 2 integers in nondecreasing order:
-2 1
Array 4 size: 1
Enter 1 integers in nondecreasing order:
0
Input: [[], [-3, 1, 1], [-2, 1], [0]]
Merged array: [-3, -2, 0, 1, 1, 1]
Process exited with status 0
```

Test Case 2: One array with duplicates

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8/problem8'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem8/problem8'
Problem 8: Merge K Sorted Arrays
Enter the number of arrays: 1
Array 1 size: 3
Enter 3 integers in nondecreasing order:
-1 -1 2
Input: [[-1, -1, 2]]
Merged array: [-1, -1, 2]
Process exited with status 0
```

Problem 9

Kth-Largest Element

Problem Analysis

Goal

Find the kth-largest element in an unsorted integer array without fully sorting it.

Input

An array of n integers and an integer k with $1 \leq k \leq n$.

Output

The kth-largest value.

Basic Conditions

Duplicate values count as separate positions. The implementation uses a min-heap of size k .

Problem 9

Algorithm Approach

Maintain a Min-Heap of the Largest k Values

My approach is to first create an empty min-heap and use an index i to traverse the entire array. The smallest value in the heap is always at the top.

Next, I insert the value at index i into the heap. If the number of elements in the heap exceeds k , I remove the value at the top, which is the smallest value currently in the heap. Otherwise, I keep all values in the heap.

I then increase i by one and repeat these operations until every element in the array has been processed. In this way, after each iteration, the heap keeps the largest k values seen so far, or all values seen so far if fewer than k have been processed.

Finally, I output the value at the top of the heap. Since the heap contains the largest k values, the smallest among them is the k th-largest element of the original array. Repeated values are stored separately and are not removed as duplicates.

Problem 9

Pseudocode

Algorithm 1 Kth-Largest Element Using a Min-Heap

Input: An integer array A of length n and k , where $1 \leq k \leq n$.

Output: The k th-largest value, counting duplicate occurrences.

```
1:  $H \leftarrow$  an empty min-heap
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:     INSERT( $H$ ,  $A[i]$ )
4:     if SIZEOF( $H$ ) >  $k$  then
5:         EXTRACT-MIN( $H$ )
6:     end if
7: end for
8: return MIN( $H$ )
```

Problem 9

Correctness Explanation

After processing each input value, the min-heap keeps the largest k values seen so far. A new value is inserted, and when the heap contains more than k values, the smallest one is removed.

Thus, after all values are processed, the heap contains the largest k values in the array. The smallest value in this heap is exactly the k th-largest value.

Duplicate values are stored separately, so they count as separate positions. The required range $1 \leq k \leq n$ ensures that the heap is nonempty when its top value is returned.

Problem 9

Time Complexity and Space Complexity

Let n be the array length and $1 \leq k \leq n$. The min-heap holds k values, so its height is $O(\log(k + 1))$.

1. Heap scan

Operation	Contribution
Initialization and return	2
For-loop control	$2n + 2$
Heap insertion	$n \log_2(k + 1)$
Heap removal	$n \log_2(k + 1)$

$$T(n, k) \leq 2 + (2n + 2) + 2n \log_2(k + 1)$$

$$T(n, k) = O(n \log(k + 1)); \text{ when } k = 1, T(n, 1) = O(n).$$

Space Complexity

The heap stores k values, so auxiliary space is $O(k)$. Including the input vector, total storage is $O(n)$.

Problem 9

Test Cases

Case / Purpose	Input	Expected Output
1. Unsorted; duplicate maximum; negatives	A = [-2,5,1,5,0]; k = 2	5
2. k = n; duplicate minimum (edge)	A = [3,-1,0,-1]; k = 4	-1
3. Single element; k = 1 (edge)	A = [-7]; k = 1	-7

Problem 9

Execution Screenshots

Both VS Code debug runs exited with status 0. The first test verifies that repeated maximum values are counted separately. The second test verifies the boundary case $k = n$, which returns the smallest value.

Test Case 1: Repeated maximum values

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9/problem9'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9/problem9'
Problem 9: Kth-Largest Element
Enter the number of elements: 5
Enter 5 integers:
-2 5 1 5 0
Enter k (1 to 5): 2
Input: [-2, 5, 1, 5, 0]
k = 2
Kth-largest element: 5
Process exited with status 0
```

Test Case 2: k equals the array length

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9/problem9'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem9/problem9'
Problem 9: Kth-Largest Element
Enter the number of elements: 4
Enter 4 integers:
3 -1 0 -1
Enter k (1 to 4): 4
Input: [3, -1, 0, -1]
k = 4
Kth-largest element: -1
Process exited with status 0
```

Problem 10

Count Inversions

Problem Analysis

Goal

Count pairs (i, j) for which $i < j$ and $A[i] > A[j]$.

Input

An integer array of length n .

Output

A 64-bit integer giving the number of inversions.

Basic Conditions

Modified Merge Sort is the primary $O(n \log n)$ solution. Brute force is included for validation.

Problem 10

Algorithm Approaches

1. Brute Force (Validation Only)

My approach is to first create a 64-bit integer variable count and initialize it to 0. I use an index i to traverse the array from the first element. For each position i , I create another index j , starting at $i + 1$, to traverse all elements after it.

If the value at index i is greater than the value at index j , I increase count by one. Otherwise, I do nothing. I increase j until it reaches the end of the array, then increase i and repeat these steps. Finally, I output count. I do not need to store the individual inversion pairs.

2. Modified Merge Sort (Primary Solution)

My approach is to first divide the array into left and right parts, then repeatedly divide each part until it contains at most one element. I recursively count the inversions inside each part and sort both parts in ascending order.

Next, I use indices i and j to traverse the two sorted parts. If the left value is less than or equal to the right value, I place the left value into a temporary array and increase i . Otherwise, I place the right value into the temporary array. Since all remaining values in the left part are greater than this right value, I add the number of remaining left values to the count and increase j .

I repeat these comparisons until one part is exhausted, append the remaining values from the other part, and copy the merged values back. I add the counts from the left part, the right part, and the merging step. Finally, I output the total count. All counts use 64-bit integers, and equal values are not counted as inversions.

Problem 10

Pseudocode

Algorithm 1 Brute-Force Inversion Count

Input: An integer array A of length n .

Output: The number of inversions, stored as a 64-bit integer.

```
1: count  $\leftarrow$  0 (64-bit integer)
2: for  $i \leftarrow 0$  to  $n - 1$  do
3:     for  $j \leftarrow i + 1$  to  $n - 1$  do
4:         if  $A[i] > A[j]$  then
5:             count  $\leftarrow$  count + 1
6:         end if
7:     end for
8: end for
9: return count
```

Problem 10

Pseudocode

Algorithm 2 Modified Merge Sort

Input: Array A; SORT-COUNT uses the half-open range [left, right).

Output: A 64-bit inversion count; the original input is preserved.

```
1: S ← COPY(A); B ← a temporary array of SIZEOF(A)
2: return SORT-COUNT(S, B, 0, SIZEOF(S))
3:
4: procedure SORT-COUNT(S, B, left, right)
5:     if right - left ≤ 1 then return 0 end if
6:     mid ← left + FLOOR((right - left) / 2)
7:     count ← SORT-COUNT(S, B, left, mid) (64-bit)
8:     count ← count + SORT-COUNT(S, B, mid, right)
9:     i ← left; j ← mid; out ← left
10:    while i < mid and j < right do
11:        if S[i] ≤ S[j] then
12:            B[out] ← S[i]; i ← i + 1
13:        else
14:            B[out] ← S[j]; j ← j + 1
15:            count ← count + (mid - i)
16:        end if
17:        out ← out + 1
18:    end while
19:    while i < mid do
20:        B[out] ← S[i]; i ← i + 1; out ← out + 1
21:    end while
22:    while j < right do
23:        B[out] ← S[j]; j ← j + 1; out ← out + 1
24:    end while
25:    for p ← left to right - 1 do
26:        S[p] ← B[p]
27:    end for
28:    return count
29: end procedure
```

Problem 10

Correctness Explanation

The Brute Force method compares every pair with $i < j$ exactly once and increases the counter exactly when $A[i] > A[j]$. Therefore, it counts every inversion once.

In Merge Sort, the recursive calls count inversions inside the left and right halves. During merging, when a right value is chosen before a left value, it forms inversions with every remaining left value, and the program adds that number to the count.

These three parts cover every inversion exactly once: inside the left half, inside the right half, or across the two halves. Empty and one-element arrays have no inversions, so the base case returns zero.

Problem 10

Time Complexity and Space Complexity

Let n be the array length. The two implemented methods are counted separately; Merge Sort is the primary solution.

1. Brute-force method

Operation	Contribution
Outer-loop control	$2n$
Inner-loop controls	$n(n - 1)/2$
Compare pair and possible counter update	$n(n - 1)$

$$T_{\text{brute}}(n) \leq 2n + 3n(n - 1)/2 \Rightarrow O(n^2)$$

2. Merge-Sort method

Operation	Contribution
Work at each recursion level	cn
Number of merge levels	$\log_2 n$

$$T(n) = nT(1) + cn \log_2 n$$

$$T_{\text{merge-sort}}(n) = O(n \log n). \text{ Running both methods for validation gives } O(n^2).$$

Space Complexity

Brute force uses $O(1)$ auxiliary space. Merge Sort uses an $O(n)$ buffer and an $O(\log n)$ recursion stack, so its auxiliary space is $O(n)$.

Problem 10

Test Cases

Use Merge Sort as the primary method; compare with Brute Force for validation.

Case / Purpose	Input	Expected Output
1. Provided example; cross-half inversions	[2,4,1,3,5]	3
2. Negatives; duplicates; mixed order	[3,-1,3,0,-1]	6
3. All equal; no inversions (edge)	[2,2,2]	0

Problem 10

Execution Screenshots

Both VS Code debug runs exited with status 0. The first test verifies the primary Merge Sort solution. The second test verifies the brute-force validation solution with repeated and negative values.

Test Case 1: Merge Sort solution

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10/problem10'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10/problem10'
Problem 10: Count Inversions
1. Merge Sort (primary solution)
2. Brute Force (validation only)
3. Run both and compare (small inputs recommended)
Choose a method: 1
Enter the number of elements: 5
Enter 5 integers:
2 4 1 3 5
Input: [2, 4, 1, 3, 5]
Merge Sort inversion count: 3
Process exited with status 0
```

Test Case 2: Brute-force validation solution

```
Launching: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10/problem10'
Working directory: '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10'
1 arguments:
argv[0] = '/Users/yuxuanwei/Desktop/fundamental computer engineering/作业1/Problem10/problem10'
Problem 10: Count Inversions
1. Merge Sort (primary solution)
2. Brute Force (validation only)
3. Run both and compare (small inputs recommended)
Choose a method: 2
Enter the number of elements: 5
Enter 5 integers:
3 -1 3 0 -1
Input: [3, -1, 3, 0, -1]
Brute Force inversion count: 6
Process exited with status 0
```